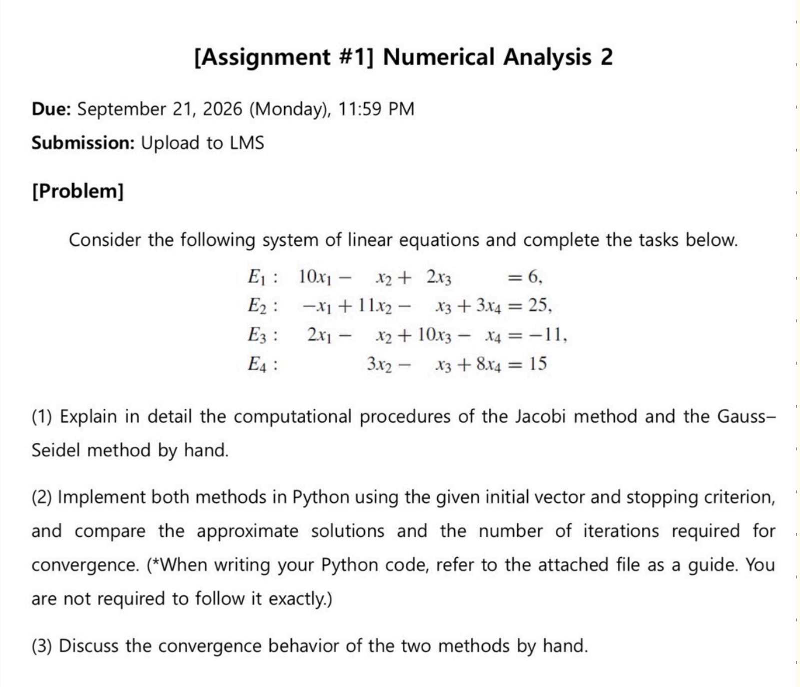


## [Assignment #1] Numerical Analysis 2

**Due:** September 21, 2026 (Monday), 11:59 PM

**Submission:** Upload to LMS

**[Problem]**

Consider the following system of linear equations and complete the tasks below.

$$\begin{aligned} E_1:&\quad 10x_1 - x_2 + 2x_3 = 6,\\ E_2:&\quad -x_1 + 11x_2 - x_3 + 3x_4 = 25,\\ E_3:&\quad 2x_1 - x_2 + 10x_3 - x_4 = -11,\\ E_4:&\quad 3x_2 - x_3 + 8x_4 = 15 \end{aligned}$$

(1) Explain in detail the computational procedures of the Jacobi method and the Gauss–Seidel method by hand.

(2) Implement both methods in Python using the given initial vector and stopping criterion, and compare the approximate solutions and the number of iterations required for convergence. (*When writing your Python code, refer to the attached file as a guide. You are not required to follow it exactly.)

(3) Discuss the convergence behavior of the two methods by hand.

이름: 김민재.

학번: 20221414

(1) (3) step 1). 연산 가능한지 확인.

1-1 비특이 행렬일 및 대각성분 존재 확인.

연립방정식 계수항에 대해 비특이행렬임을 확인한다.

$AX = b$ , $|A| \neq 0$?

$$\begin{bmatrix} 10 & -1 & 2 & 0\\ -1 & 11 & -1 & 3\\ 2 & -1 & 10 & -1\\ 0 & 3 & -1 & 8 \end{bmatrix} = A,$$

$$|A| = 10\begin{vmatrix} 11 & -1 & 3\\ -1 & 10 & -1\\ 3 & -1 & 8 \end{vmatrix} + 1\begin{vmatrix} -1 & -1 & 3\\ 2 & 10 & -1\\ 0 & -1 & 8 \end{vmatrix} + 2\begin{vmatrix} -1 & 11 & 3\\ 2 & -1 & -1\\ 0 & 3 & 8 \end{vmatrix} - 0\begin{vmatrix} -1 & 11 & -1\\ 2 & -1 & 10\\ 0 & 3 & -1 \end{vmatrix}$$

$$10\begin{pmatrix} \overbrace{880+3+3}^{886}\\ -\\ \underbrace{70+8+11}_{107} \end{pmatrix} + \begin{pmatrix} -80+0-6\\ -\\ 0-16-1 \end{pmatrix} + 2\begin{pmatrix} 8+0+18\\ -\\ 0+66+3 \end{pmatrix} \quad 0$$

$$\begin{array}{r} 886\\ 10\times\ -107\\ \hline 779 \end{array} \qquad \begin{array}{r} -86\\ +17\\ \hline -69 \end{array} \qquad 2\times\begin{array}{r} 26\\ +69\\ \hline 95 \end{array}$$

$$\begin{array}{r} 7790\\ 111\\ \hline 7881 \end{array} \qquad 7790\ \underbrace{-69 \qquad +190}_{111} = 7881 \neq 0$$

=> 계수행렬이 비특이행렬이다.

=> 단 하나의 유일해 $X$ 가 존재한다.

$a_{11} \neq 0$ , $a_{22} \neq 0$, $a_{33} \neq 0$, $a_{44} \neq 0$ 모든 대각성분이 0이 아니다.

=> 각 성분별 오차 수렴여부 계산할 수 있다!

1-2. 수렴 여부 확인.

Jacobi 반복법 수행시 근사해가 유일해 $X$로 수렴하도록 오차가 지속적으로 수렴하는지 알아본다.

연립방정식. $AX = b$ 에 대하여.

A에 대해 $U$; 상삼각행렬 $L$; 하삼각행렬 , $D$; 대각행렬 이라 할 때.

-1 이 곱해진 형태

$A = D - L - U$

$AX = (D - L - U)X = b$ 가 된다.

=>

$DX = (L+U)X + b$

$$\overset{\textcircled{1}}{\underline{X}} = \underbrace{D^{-1}\cdot(L+U)}_{T}\overset{\textcircled{2}}{\underline{X}} + \underbrace{D^{-1}b}_{C}$$

이때.

T의 경우 대각성분이 0이 된 행렬로. $TX + C$ 를 통해 구해진 $X$ 를. 각 행에 대한 대각성분의 요소의 $X$가 되어진다.

Jacobi 반복기서는.

→ 이때. 임의의 $X_0$ 에 대해.

$TX_0 - C$ 를 통해 도출된 $X_1$ 이 $X_0$보다 유일해 $X$에 대해 더 근사하고 있어야 한다.

즉 $|X - X_1| < |X - X_0|$ 이어야 한다.

$E_n = |X - X_n|$ 에 대해서 n번 반복해얻은 해의 오차라 정의 할 때

$X_n = TX_{n-1} - C$ 에 대해. $X = TX - C$ 에서 빼서 주어

$$\begin{aligned} X - X_n &= TX - C - (TX_n - C)\\ &= T(X - X_n) \end{aligned}$$

$E_n$을. 대입시. $E_n$의 스케일 변화 값을 확인 하기 위함으로. $(X - X_n)$, $-(X - X)$ 두 케이스 모두 확인 X.

↳ 수식에 대해 $|X - X_n| = T|X - X_n|$ 로 두고 전개한다.

$E_n = TE_{n-1}$ 이라고 하자.

이때. $E_n$ 은. T의 변환에 따른 스케일변화로서 수렴,발산 여부를 확인할수 있는데

$$E_n = (\underbrace{T\cdot T\cdots T}_{n-1})E_0 = T^nE_0.$$

↳. 즉 T행렬에 대하여 고유값 λ가 $P(T) = \max|\lambda|$ 라 할 때.

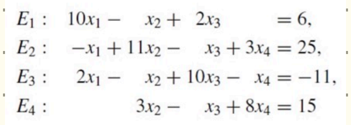
*$P(T) < 1$

$\lim_{n\to\infty} T^nE_0 = 0$ 로. $P(T) < 1$ 에서.

반복횟수가 증가함에 따라 오차가 0으로 수렴한다.

=>

2026-Fall Numerical Analysis II
Week3 pdf
Theorem 7.17.



$$\begin{aligned} E_1 &: 10x_1 - x_2 + 2x_3 = 6, \\ E_2 &: -x_1 + 11x_2 - x_3 + 3x_4 = 25, \\ E_3 &: 2x_1 - x_2 + 10x_3 - x_4 = -11, \\ E_4 &: 3x_2 - x_3 + 8x_4 = 15 \end{aligned}$$

에 대한 Jacobi 반복에 따른 오차감소여부를 알아본다.

$T = D^{-1}(L+U)$

$$T = \begin{bmatrix} 0 & +\frac{1}{10} & -\frac{2}{10} & 0 \\ +\frac{1}{11} & 0 & +\frac{1}{11} & -\frac{3}{11} \\ -\frac{2}{10} & +\frac{1}{10} & 0 & +\frac{1}{10} \\ 0 & -\frac{3}{8} & +\frac{1}{8} & 0 \end{bmatrix}$$

$\det(T - \lambda I) = 0.$

T 에서 확인할수 있듯이.

각 원소는.

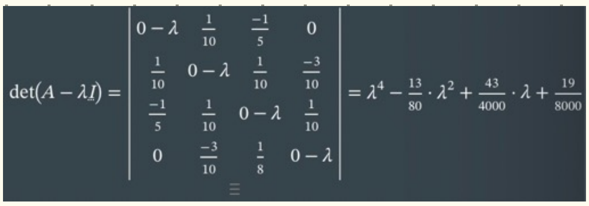
$\frac{a_{ij}(i \neq j)}{a_{ii}}$ 으로 이루어져 있음 을알 수 있다.

이때. $a_{ii} > a_{ij}(i \neq j)$ 일수록, 즉 대각행렬이 해당하는 행의 나머지기 비해 우세할 때

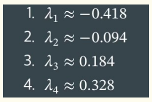
T 행렬의 각원소의 크기가 작아져

$|\lambda|$값이 크기 또한 이에 비례하는 모습을 알수있다.

matrixcalc.org. 고유값 계산기 결과.



$$\det(A - \lambda I) = \begin{vmatrix} 0-\lambda & \frac{1}{10} & \frac{-1}{5} & 0 \\ \frac{1}{10} & 0-\lambda & \frac{1}{10} & \frac{-3}{10} \\ \frac{-1}{5} & \frac{1}{10} & 0-\lambda & \frac{1}{10} \\ 0 & \frac{-3}{10} & \frac{1}{8} & 0-\lambda \end{vmatrix} = \lambda^4 - \frac{13}{80}\cdot\lambda^2 + \frac{43}{4000}\cdot\lambda + \frac{19}{8000}$$



1. $\lambda_1 \approx -0.418$
2. $\lambda_2 \approx -0.094$
3. $\lambda_3 \approx 0.184$
4. $\lambda_4 \approx 0.328$

$\rho(T) = \max|\lambda|$ 이므로

$\rho(T) = 0.328 < 1$ 이므로.

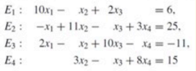
$AX = b$ 의 Jacobi 반복법에 대해.

$n \to \infty$ 가 될때 근사해 $X_n$는 유일해 $X$ 으로 수렴 함을알 수있다.

step1 을 통해 유일해 $X$가 존재하고.

Jacobi 반복법을통해 근사해 $X_n$이 유일해 $X$ 에 수렴함을 알수있었다.

## Step 2. 반복법을 통한 근사해 구하기.



$$\begin{aligned} E_1 &: 10x_1 - x_2 + 2x_3 = 6, \\ E_2 &: -x_1 + 11x_2 - x_3 + 3x_4 = 25, \\ E_3 &: 2x_1 - x_2 + 10x_3 - x_4 = -11, \\ E_4 &: 3x_2 - x_3 + 8x_4 = 15 \end{aligned}$$

$X_0 = \{x_1^0, x_2^0, x_3^0, x_4^0\} = \{0, 0, 0, 0\}$

- Jacobi 반복법을 통한 근사

=> 1.

$$\begin{aligned} x_1^{(1)} &= +\frac{1}{10}x_2^{(0)} - \frac{2}{10}x_3^{(0)} + \frac{6}{10} = \frac{6}{10} \\ x_2^{(1)} &= \frac{1}{11}x_1^{(0)} + \frac{1}{11}x_3^{(0)} + \frac{3}{11}x_4^{(0)} + \frac{25}{11} = \frac{25}{11} \\ x_3^{(1)} &= -\frac{2}{10}x_1^{(0)} + \frac{1}{10}x_2^{(0)} + \frac{1}{10}x_4^{(0)} - \frac{11}{10} = -\frac{11}{10} \\ x_4^{(1)} &= -\frac{3}{8}x_2^{(0)} + \frac{1}{8}x_3^{(0)} + \frac{15}{8} = \frac{15}{8} \end{aligned}$$

=> 2

$$\begin{aligned} x_1^{(2)} &= +\frac{1}{10}x_2^{(1)} - \frac{2}{10}x_3^{(1)} + \frac{6}{10} = \frac{1}{10}\cdot\frac{25}{11} + \frac{2}{10}\frac{11}{10} + \frac{6}{10} \approx 1.047 \\ x_2^{(2)} &= \frac{1}{11}x_1^{(1)} + \frac{1}{11}x_3^{(1)} + \frac{3}{11}x_4^{(1)} + \frac{25}{11} = \frac{1}{11}\frac{6}{10} - \frac{1}{11}\cdot\frac{11}{10} + \frac{3}{11}\cdot\frac{15}{8} + \frac{25}{11} \approx 2.73 \\ x_3^{(2)} &= -\frac{2}{10}x_1^{(1)} + \frac{1}{10}x_2^{(1)} + \frac{1}{10}x_4^{(1)} - \frac{11}{10} = -\frac{2}{10}\cdot\frac{6}{10} + \frac{1}{10}\cdot\frac{25}{11} + \frac{1}{10}\cdot\left(-\frac{11}{10}\right) + \frac{1}{10}\cdot\frac{15}{8} - \frac{11}{10} \approx -0.915 \\ x_4^{(2)} &= -\frac{3}{8}x_2^{(1)} + \frac{1}{8}x_3^{(1)} + \frac{15}{8} = -\frac{3}{8}\cdot\frac{25}{11} + \frac{1}{8}\left(-\frac{11}{10}\right) + \frac{15}{8} \approx 0.885 \end{aligned}$$

- Gauss-Seidel 반복법.

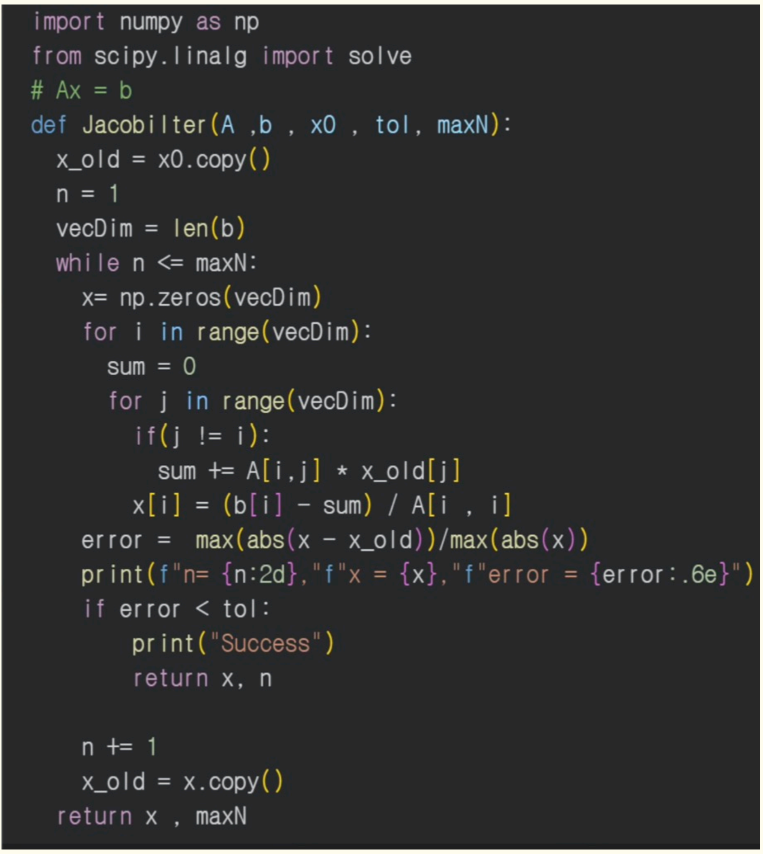
=> 1.

$$\begin{aligned} x_1^{(1)} &= +\frac{1}{10}x_2^{(0)} - \frac{2}{10}x_3^{(0)} + \frac{6}{10} = \frac{6}{10} = 0.6 \\ x_2^{(1)} &= \frac{1}{11}x_1^{(1)} + \frac{1}{11}x_3^{(0)} + \frac{3}{11}x_4^{(0)} + \frac{25}{11} = \frac{1}{11}\cdot\frac{6}{10} + \frac{25}{11} \approx 2.327 \\ x_3^{(1)} &= -\frac{2}{10}x_1^{(1)} + \frac{1}{10}x_2^{(1)} + \frac{1}{10}x_4^{(0)} - \frac{11}{10} = -\frac{2}{10}\cdot\frac{6}{10} + \frac{1}{10}(2.327) - \frac{11}{10} \approx -0.987. \\ x_4^{(1)} &= -\frac{3}{8}x_2^{(1)} + \frac{1}{8}x_3^{(1)} + \frac{15}{8} = -\frac{3}{8}\cdot 2.327 + \frac{1}{8}\cdot(-0.987) + \frac{15}{8} \approx 0.877 \end{aligned}$$

=>.

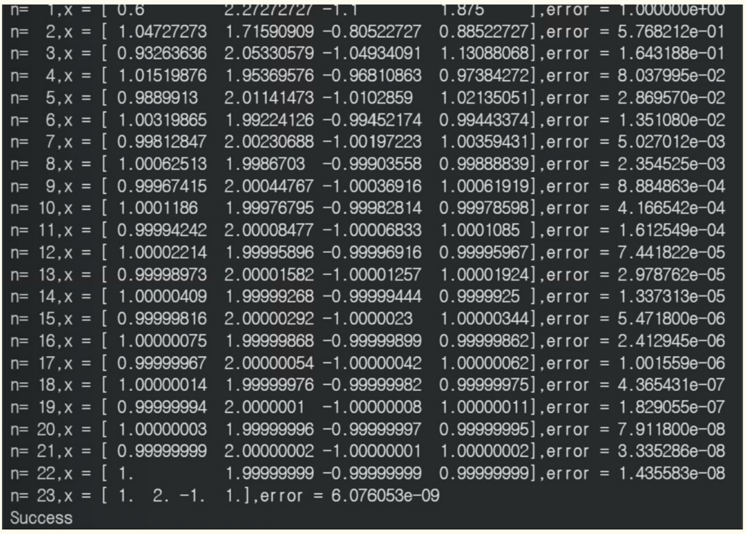
$$\begin{aligned} x_1^{(2)} &= +\frac{1}{10}x_2^{(1)} - \frac{2}{10}x_3^{(1)} + \frac{6}{10} \approx 0.854 \\ x_2^{(2)} &= \frac{1}{11}x_1^{(2)} + \frac{1}{11}x_3^{(1)} + \frac{3}{11}x_4^{(1)} + \frac{25}{11} \approx 2.5 \\ x_3^{(2)} &= -\frac{2}{10}x_1^{(2)} + \frac{1}{10}x_2^{(2)} + \frac{1}{10}x_4^{(1)} - \frac{11}{10} \approx -1.119 \\ x_4^{(2)} &= -\frac{3}{8}x_2^{(2)} + \frac{1}{8}x_3^{(2)} + \frac{15}{8} \approx 0.797 \end{aligned}$$

## Jacobi 반복법 Code



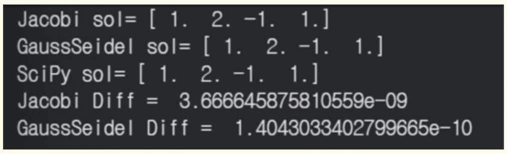
```
import numpy as np
from scipy.linalg import solve
# Ax = b
def JacobiIter(A ,b , x0 , tol, maxN):
  x_old = x0.copy()
  n = 1
  vecDim = len(b)
  while n <= maxN:
    x= np.zeros(vecDim)
    for i in range(vecDim):
      sum = 0
      for j in range(vecDim):
        if(j != i):
          sum += A[i,j] * x_old[j]
        x[i] = (b[i] - sum) / A[i , i]
    error =  max(abs(x - x_old))/max(abs(x))
    print(f"n= {n:2d},"f"x = {x},"f"error = {error:.6e}")
    if error < tol:
        print("Success")
        return x, n

    n += 1
    x_old = x.copy()
  return x , maxN
```

## Jacobi 반복법 결과



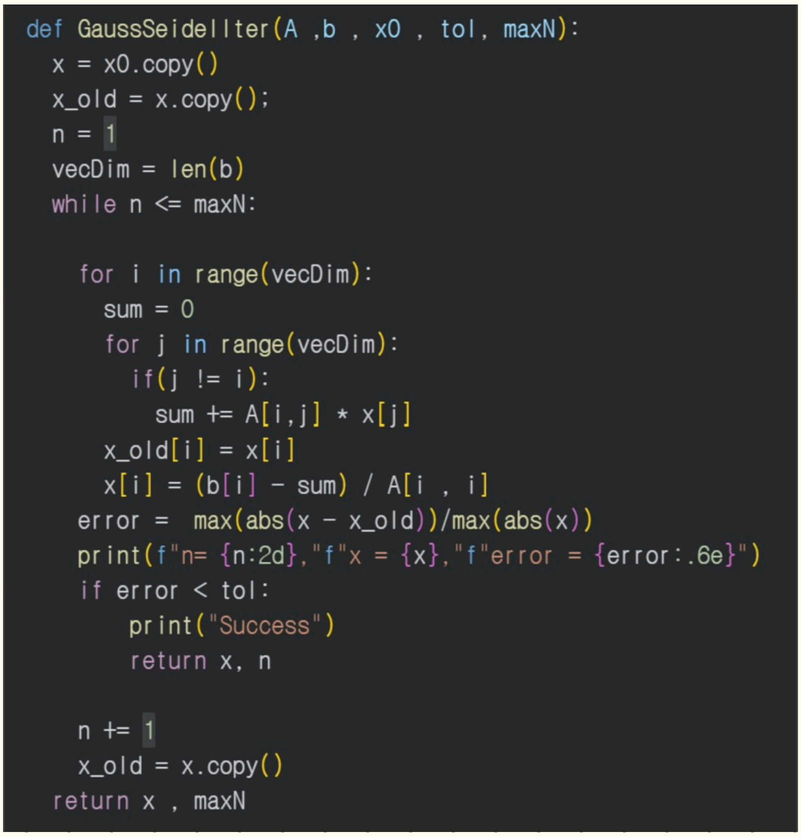
```
n=  1,x = [ 0.6         2.27272727 -1.1         1.875     ],error = 1.000000e+00
n=  2,x = [ 1.04727273  1.71590909 -0.80522727  0.88522727],error = 5.768212e-01
n=  3,x = [ 0.93263636  2.05330579 -1.04934091  1.13088068],error = 1.643188e-01
n=  4,x = [ 1.01519876  1.95369576 -0.96810863  0.97384272],error = 8.037995e-02
n=  5,x = [ 0.9889913   2.01141473 -1.0102859   1.02135051],error = 2.869570e-02
n=  6,x = [ 1.00319865  1.99224126 -0.99452174  0.99443374],error = 1.351080e-02
n=  7,x = [ 0.99812847  2.00230688 -1.00197223  1.00359431],error = 5.027012e-03
n=  8,x = [ 1.00062513  1.9986703  -0.99903558  0.99888839],error = 2.354525e-03
n=  9,x = [ 0.99967415  2.00044767 -1.00036916  1.00061919],error = 8.884863e-04
n= 10,x = [ 1.0001186   1.99976795 -0.99982814  0.99978598],error = 4.166542e-04
n= 11,x = [ 0.99994242  2.00008477 -1.00006833  1.0001085 ],error = 1.612549e-04
n= 12,x = [ 1.00002214  1.99995896 -0.99996916  0.99995967],error = 7.441822e-05
n= 13,x = [ 0.99998973  2.00001582 -1.00001257  1.00001924],error = 2.978762e-05
n= 14,x = [ 1.00000409  1.99999268 -0.99999444  0.9999925 ],error = 1.337313e-05
n= 15,x = [ 0.99999816  2.00000292 -1.0000023   1.00000344],error = 5.471800e-06
n= 16,x = [ 1.00000075  1.99999868 -0.99999899  0.99999862],error = 2.412945e-06
n= 17,x = [ 0.99999967  2.00000054 -1.00000042  1.00000062],error = 1.001559e-06
n= 18,x = [ 1.00000014  1.99999976 -0.99999982  0.99999975],error = 4.365431e-07
n= 19,x = [ 0.99999994  2.0000001  -1.00000008  1.00000011],error = 1.829055e-07
n= 20,x = [ 1.00000003  1.99999996 -0.99999997  0.99999995],error = 7.911800e-08
n= 21,x = [ 0.99999999  2.00000002 -1.00000001  1.00000002],error = 3.335286e-08
n= 22,x = [ 1.          1.99999999 -0.99999999  0.99999999],error = 1.435583e-08
n= 23,x = [ 1.  2. -1.  1.],error = 6.076053e-09
Success
```

## 결과.



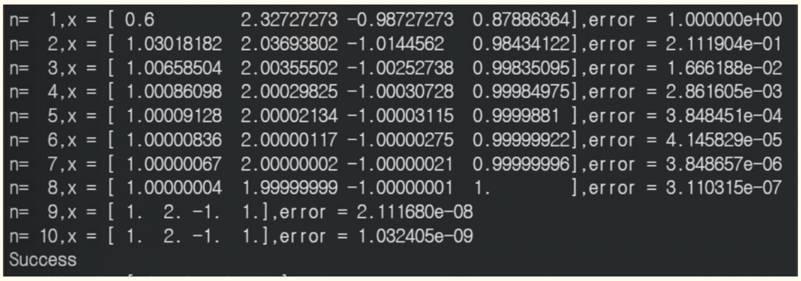
```
Jacobi sol= [ 1.  2. -1.  1.]
GaussSeidel sol= [ 1.  2. -1.  1.]
SciPy sol= [ 1.  2. -1.  1.]
Jacobi Diff =  3.666645875810559e-09
GaussSeidel Diff =  1.4043033402799665e-10
```

## GaussSeidel 반복법 Code.



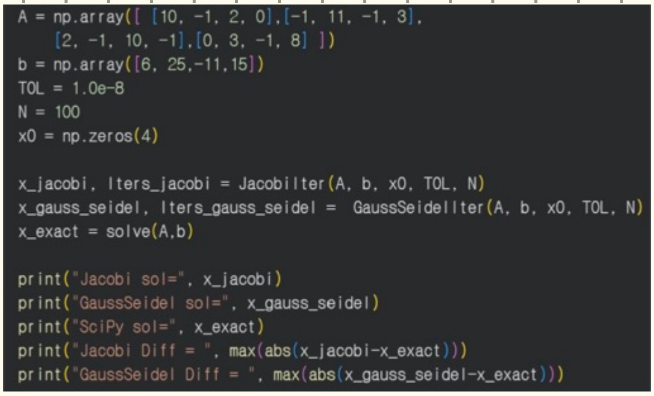
```
def GaussSeidelIter(A ,b , x0 , tol, maxN):
  x = x0.copy()
  x_old = x.copy();
  n = 1
  vecDim = len(b)
  while n <= maxN:

    for i in range(vecDim):
      sum = 0
      for j in range(vecDim):
        if(j != i):
          sum += A[i,j] * x[j]
      x_old[i] = x[i]
      x[i] = (b[i] - sum) / A[i , i]
    error =  max(abs(x - x_old))/max(abs(x))
    print(f"n= {n:2d},"f"x = {x},"f"error = {error:.6e}")
    if error < tol:
        print("Success")
        return x, n

    n += 1
    x_old = x.copy()
  return x , maxN
```

## GaussSeidel 반복법. 결과



```
n=  1,x = [ 0.6         2.32727273 -0.98727273  0.87886364],error = 1.000000e+00
n=  2,x = [ 1.03018182  2.03693802 -1.0144562   0.98434122],error = 2.111904e-01
n=  3,x = [ 1.00658504  2.00355502 -1.00252738  0.99835095],error = 1.666188e-02
n=  4,x = [ 1.00086098  2.00029825 -1.00030728  0.99984975],error = 2.861605e-03
n=  5,x = [ 1.00009128  2.00002134 -1.00003115  0.9999881 ],error = 3.848451e-04
n=  6,x = [ 1.00000836  2.00000117 -1.00000275  0.99999922],error = 4.145829e-05
n=  7,x = [ 1.00000067  2.00000002 -1.00000021  0.99999996],error = 3.848657e-06
n=  8,x = [ 1.00000004  1.99999999 -1.00000001  1.        ],error = 3.110315e-07
n=  9,x = [ 1.  2. -1.  1.],error = 2.111680e-08
n= 10,x = [ 1.  2. -1.  1.],error = 1.032405e-09
Success
```

## 실행 코드.



```
A = np.array([ [10, -1, 2, 0],[-1, 11, -1, 3],
    [2, -1, 10, -1],[0, 3, -1, 8] ])
b = np.array([6, 25,-11,15])
TOL = 1.0e-8
N = 100
x0 = np.zeros(4)

x_jacobi, Iters_jacobi = JacobiIter(A, b, x0, TOL, N)
x_gauss_seidel, Iters_gauss_seidel =  GaussSeidelIter(A, b, x0, TOL, N)
x_exact = solve(A,b)

print("Jacobi sol=", x_jacobi)
print("GaussSeidel sol=", x_gauss_seidel)
print("SciPy sol=", x_exact)
print("Jacobi Diff = ", max(abs(x_jacobi-x_exact)))
print("GaussSeidel Diff = ", max(abs(x_gauss_seidel-x_exact)))
```

위와 같이 TOL : 1.0e-8 이내로 수렴 까지의 반복결과 연립 방정식 문제에 대해 연산이 끝난뒤 next 근사값을 업데이트 하는 Jacobi 반복에 비해 연립방정식의 연산 과정에서 먼저 구한 미지수의 근사값을 즉시 최신의 값으로 업데이트 하는 Gauss-seidel 방식이 더욱 빠르게 수렴 할을 알 수 있다.